Java Functions (Methods)

In Java, **functions** are referred to as **methods**. They are blocks of code designed to perform a specific task and are invoked by name when needed.

1. Declaring and Defining a Method

```
returnType methodName(parameter1, parameter2, ...) {
    // method body
    return value; // if returnType is not void
}
```

Example:

```
public int add(int a, int b) {
    return a + b;
}
```

2. Calling a Method

Methods are called from another method (usually `main`) using an object or class reference.

Example:

```
public class Calculator {
    public int add(int x, int y) {
        return x + y;
    }

    public static void main(String[] args) {
        Calculator calc = new Calculator();
        int result = calc.add(5, 3);
        System.out.println("Sum is: " + result);
    }
}
```

---

3. Types of Methods

Type	Description
Static method	Called without creating an object
Instance (non-static) method	Requires an object of the class
Void method	Returns nothing
Return-type method	Returns a value (e.g., int, double, String)

Static Method Example:

```
public class Example {
    static void greet() {
        System.out.println("Hello, world!");
    }
```

```
    public static void main(String[] args) {
        greet();  // no object needed
    }
}
```

4. Parameter Passing

a) By Value (Java's default)

Java passes arguments by value, meaning it passes a copy of the variable.

```
public void greetUser(String name) {
    System.out.println("Hello " + name);
}
```

b) Returning Values

```
public double square(double x) {
    return x * x;
}
```

---

5. Method Overloading

You can define multiple methods with the same name but different parameter lists.

```
java
CopyEdit
public class Print {
    void show(int a) {
        System.out.println("Int: " + a);
    }

    void show(String s) {
        System.out.println("String: " + s);
    }
}
```

6. `main()` Method

The starting point of every Java program.

```
public static void main(String[] args) {
    // Code to execute
}
```

Java Arrays

Arrays are used to store multiple values of the same type in a single variable, instead of declaring separate variables for each value.

Declaring and Creating Arrays

a) One-Dimensional Array

```
int[] numbers = new int[5];  // declaration and memory allocation
numbers[0] = 10;             // assigning value
```

b) Declaration with Initialization

```
String[] fruits = {"Apple", "Banana", "Mango"};
```

Accessing and Modifying Array Elements

```
System.out.println(fruits[1]);  // Output: Banana
fruits[2] = "Orange";           // Update element at index 2
```

Traversing Arrays Using Loops

a) Using `for` Loop

```
int[] scores = {70, 85, 90, 60, 88};

for (int i = 0; i < scores.length; i++) {
    System.out.println("Score " + i + ": " + scores[i]);
}
```

Multidimensional Arrays

a) Declaration and Initialization

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};
```

b) Accessing Elements

```
System.out.println(matrix[0][2]); // Output: 3
```

c) Traversing with Nested Loops

```
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}
```

5. Array Class Utilities (Optional Intro)

```
import java.util.Arrays;

int[] data = {5, 3, 8, 1};
Arrays.sort(data);  // Sort array
System.out.println(Arrays.toString(data));  // Print array
```